CATÓLICA
LISBON
BUSINESS & ECONOMICS

Track B: BirdCLEF+ 2026

Bird Species Recognition from Audio — Autonomous Research Agent

Advanced Predictive Analytics 2025/2026

Diogo Ribeiro 800535855
Pedro Fernandes 800536241
Sofia Póvoa 800535764

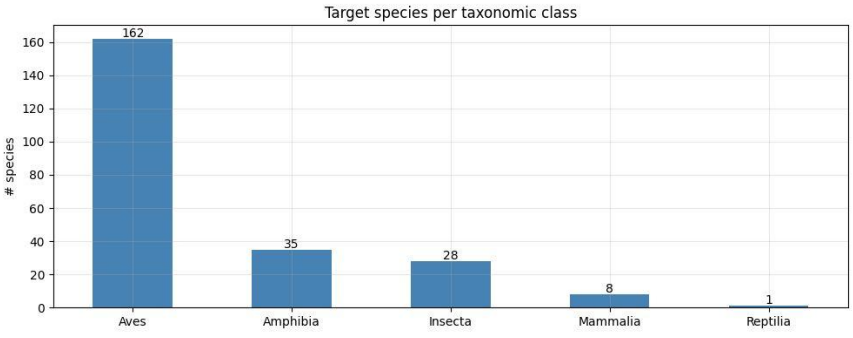
1. Introduction

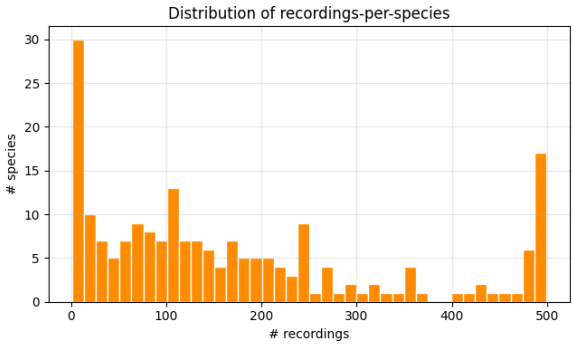
The Kaggle competition chosen by our group was BirdCLEF+ 2026 (Track B), which involves recognising bird species from field recordings collected across the Pantanal wetlands. Bird species recognition from audio is an important task for biodiversity monitoring, but manual identification is slow, expert-dependent, and difficult to scale. This project addresses the BirdCLEF+ 2026 challenge by developing an autonomous research agent for multi-label bird classification from soundscape recordings. The goal is to evaluate whether this agent-based approach can improve performance over a simple baseline, with particular focus on validation AUC, Kaggle leaderboard results, and the gap between clip-based validation and real soundscape performance.

2.Exploring the data

The dataset contains 35,549 focal recordings across the 234 target species, plus 1,478 labelled 5-second soundscape windows — the distribution we are actually scored on. Four findings shaped our approach:

- Not just birds - Targets span Aves, Amphibia, Insecta, Mammalia and Reptilia. Those are acoustically very different signals that one shared spectrogram model must handle.
- Severe class imbalance - Recordings per species range from 1 to 499 (median 125), and ~25% of species have fewer than 50. Since scoring is macro ROC-AUC, this rare long tail dominates the final score;
- Genuinely multi-label - 12% of focal clips carry background species, and soundscape windows average 4.2 species each;
- Domain shift is the core challenge - Test audio comes only from the Pantanal, yet just 2.4% of focal clips were recorded there and only 75 of 234 species ever appear in the labelled soundscapes.





2.1 Transformation of the sounds

Since the task is based on bird sound recordings, the original audio files could not be used directly as image-based inputs. Therefore, the main data transformation step was the conversion of each audio clip into a mel-spectrogram, which represents the audio signal in terms of frequency and time. This allowed the model to treat each recording as a visual pattern while still preserving important acoustic information.

2.2 Clean sounds dataset

This corresponds to the dataset of focal recordings. In the 35,549 clips that make up the bulk of the training data. Each clip captures a labeled single target recorded close to the source so the call of interest is clear and dominant. These are crowd-sourced from two community archives and include a 0–5 crowd quality rating. Audio is stored as .ogg files at a 32 kHz sample rate. Crucially, although these clips are clean and easy to label, they were recorded worldwide rather than in the Pantanal test region, which is the source of the domain shift discussed above.

2.3 Sounds scape dataset

Another important part of the EDA was the analysis of the soundscape dataset. Unlike clean focal recordings, soundscapes are longer and noisier field recordings that may contain multiple species, background noise, and overlapping calls. This distinction was important because the competition test data is based on real soundscape recordings, meaning that validation based only on clean clips could give overly optimistic results. As a result, the EDA helped us identify the gap between focal-clip data and soundscape data, which later influenced the validation strategy used in the project.

3. Our approach

3.1 Agent Architecture

The goal of the project was not only to train a bird classifier, but also to build a machine that builds machines: an agent loop where a Local Large Language Model (LLM) proposes a neural-network architecture, the system executes and trains it, measures its performance, and feeds the result back to the LLM so it can propose a better one on the next iteration.

The agent was driven by the Gemma 4 LLM, served locally through Ollama and we ran experiments with two pre-trained backbones: EfficientNet and YAMNet to compare their suitability for the task and results.

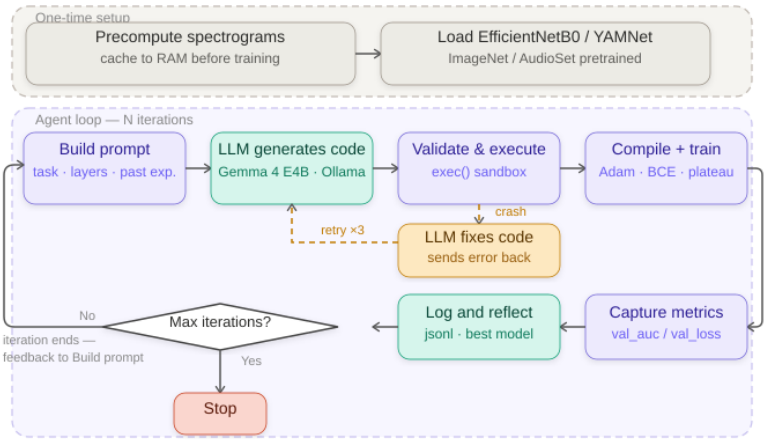
3.2 The Agent Loop

The agent runs a set number of iterations of the following loop with early stopping. Each iteration begins with prompt construction, where prompt_builder.py assembles a structured prompt describing the fixed backbone, the available Keras layers, optional training overrides and the full history of past experiments with their code and scores. This prompt is sent to Gemma 4 E4B, which returns Python code for a Keras head inside a fenced code block.

Before execution, the code is validated where any forbidden calls are stripped defensively. The validated code is then executed in a restricted namespace that exposes only the approved set of Keras layers, backbone_model, TensorFlow and NumPy. The harness compiles the resulting model with Adam, binary cross-entropy and AUC, then trains it.

After training, the LLM is asked to reflect on the result and suggest what to change next. This analysis is stored alongside the metrics. Every iteration, success or crash, is then appended as a new line to a experiments file. If the run achieved a new global best AUC, the model weights are saved to for use in the Kaggle submission notebook. The accumulated history is fed back into the next iteration's prompt, and the cycle repeats.

After all iterations the system writes a per-run report folder containing each iteration's generated code, the best model and a training curves plot of loss and AUC across iterations.



3.3 Fixed Infrastructure

Audio preprocessing begins by loading each clip at 32 kHz for a duration of 5 seconds using librosa, padding any sequences shorter than the target window. These signals are converted into a 64-bin mel-spectrogram (fmax = 16 kHz, hop length 512), yielding a (64 × 313) matrix per clip.

To eliminate disk I/O as a per-epoch bottleneck and substantially reduce training duration, all spectrograms are pre-computed once into RAM before training begins via the precompute_spectrograms() routine.

To prevent overfitting and artificially low loss surfaces, dynamic data augmentation is applied exclusively during the training phase. Waveform-level augmentations include the injection of Gaussian noise, random gain adjustments (0.7× to 1.3×), and random time shifts (±0.5 s). At the spectrogram level, SpecAugment is leveraged via frequency masking (2 masks, up to 10 bins) and time masking (2 masks, up to 40 steps). Furthermore, mix-up augmentation is applied to clip batches, utilizing Beta-blended sample pairs and soft labels to synthetically reproduce the multi-species overlap characteristic of real soundscapes.

Severe class imbalance within the dataset is mitigated by oversampling rare bird species to a strict minimum threshold of 50 examples, while species with fewer than 2 total samples are routed entirely into the training split to systematically avoid downstream stratification failures.

Finally, execution integrity is governed by a dual validation regime (toggle) infrastructure. When SOUNDSCAPE_VAL = False, validation is conducted on held-out focal clips, which offers a fast but highly optimistic performance signal. Conversely, setting SOUNDSCAPE_VAL = True forces an 80/20 split directly on the soundscape windows, forming a held-out set that perfectly aligns the training-time validation metric with the real competition scoring distribution. This toggle represented the single most impactful infrastructure change in the project, shifting the engineering focus toward realistic, runnable production metrics.

3.4 Prompt Design

The prompt is structured in five distinct sections to guide the model's generation process. First, the task description establishes the baseline architectural constraints, explicitly defining the backbone name, the required input shape (64 × 313 × 1), and the final output shape as a 1280-dimensional flat feature vector.

Second, a curated namespace of available Keras layers—including Dense, Dropout, BatchNormalization, Conv1D, MultiHeadAttention, GlobalAveragePooling1D, Reshape, Concatenate, and Multiply—is provided. This ensures the LLM knows exactly which components are supported out of the box without requiring external imports.

Third, the prompt introduces optional override variables that the LLM can manipulate. These allow the agent to dynamically set parameters such as the learning_rate, implement a custom loss function like BinaryFocalCrossentropy, or define fine_tune_layers to unfreeze the top N layers of the backbone.

Fourth, a "Techniques Worth Trying" block serves as a strategic guide for the model. This section highlights advanced methodologies that have proven effective in similar tasks, advertising the use of focal loss, squeeze-and-excitation blocks, and backbone fine-tuning.

Fifth and finally, the prompt appends the full historical log of all past experiments, sorted in descending order by val_auc. This includes the first 200 characters of each run's generated code, instructing the LLM to analyze the historical context, propose a novel architecture not yet present in the logs, and document its rationale in a reasoning comment at the top of the code block.

3.5 Error Handling and Self-Repair

If the LLM-generated code crashes syntax error, shape mismatch or a Keras contract violation, the agent enters a retry loop governed with a limit of 3. The crash error message and the failing code are sent back to the LLM with an explicit request to fix the issue, the corrected code is then re-executed.

Separately, LLM-call failures such as an Ollama connection hiccup are caught and logged as skipped iterations, so a multi-hour run can continue uninterrupted even if the LLM is briefly unavailable.

4. Experiment Log Analysis

The agent was executed across three primary runs for each backbone, resulting in six logged experiments. To ensure data integrity, no records are ever deleted or overwritten.

4.1 The Debugging Process

The true engineering value of this project lies within its debugging path, where the overarching objective was to transition from a fragile, error-prone setup to a bulletproof pipeline capable of running reliably and autonomously. Because an autonomous research agent must execute extensive training loops without human intervention, identifying and resolving runtime bottlenecks was critical to achieving operational stability. We addressed these execution blocks, categorizing the challenges into three core domains: framework and environment incompatibilities, data integrity and metric mismatches, and agent execution robustness.

Environment fragility: subtle version and serialization mismatches between our local setup and Kaggle's frozen environment could silently degrade a model to a near-random score rather than fail loudly — solved by pinning to Kaggle's exact stack and avoiding components that don't reload cleanly.

Deceptive metrics: misaligned labels and a too-forgiving default AUC made the model look excellent while it learned little, so we cleaned the label pipeline and validated against the competition's true macro objective.

Agentic robustness: since an LLM writes the code unsupervised, we hardened the loop with validation, self-correcting retries, and fault tolerance so a single bad generation or API hiccup can't derail a multi-hour run.

4.2 Crashes and self-correction

LLM-generated code does not always run on the first attempt, so each proposed head passes through an execute-and-repair loop: when the code throws, the traceback is fed back to Gemma 4, which patches it, and crash_count records how many repairs an iteration needed before it trained.

The two backbones behaved very differently here. On the EfficientNet runs the agent was largely clean only **5 of 25 iterations** crashed at all, requiring just **6 repair attempts** in total with at most two on any one iteration. The YAMNet runs were far more fragile: **21 of 23 iterations** needed at least one repair and most hit close to the three-attempt ceiling, totalling **61 repairs**, a sign that the LLM found YAMNet's input contract considerably harder to satisfy than EfficientNet's. Crucially, **every crash was recovered**: no iteration was lost and no overnight run was ever aborted, which is precisely what allows the agent to run unattended.

5. Results and Submissions

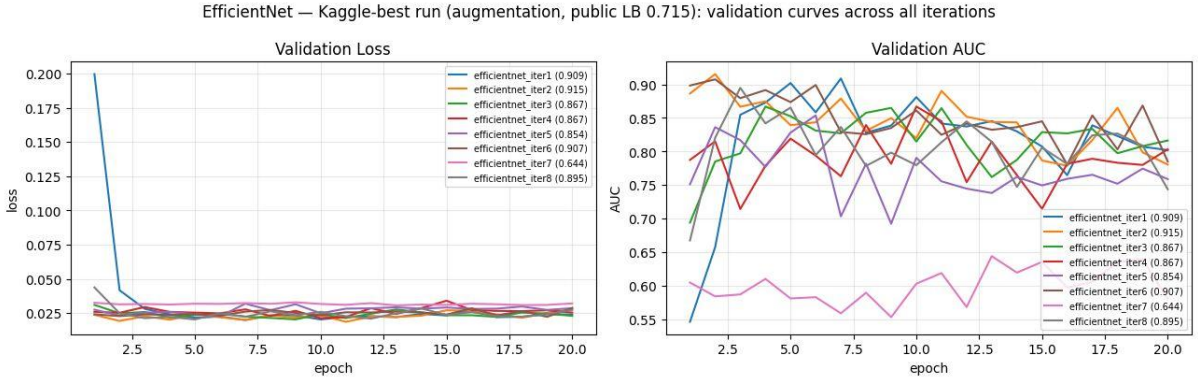
5.1 Backbone EfficientNet

The EfficientNet branch was the main image-based pipeline of the project. Audio clips were converted into 64 bin mel-spectrograms and passed through an EfficientNetB0 backbone pretrained on ImageNet. The LLM was only allowed to design the classification head on top of the extracted 1280-dimensional feature vector, while the surrounding infrastructure handled preprocessing, compilation, training, validation, logging and model saving.

In offline validation, EfficientNet produced the strongest results of the project. Simple Dense + Dropout heads already reached high validation AUC, confirming that transfer learning from a pretrained CNN was a strong baseline for mel-spectrogram classification. The agent then explored several head architectures, including Squeeze-and-Excitation, Conv1D feature refinement, multi-pathway fusion and

grouped feature decomposition. The best offline runs reached approximately 0.93 validation AUC, with grouped feature decomposition and lightweight convolutional heads among the most effective designs.

However, these high offline scores did not directly translate to Kaggle performance because clip-based validation overestimated generalisation to noisy Pantanal soundscapes. The most important improvement was therefore not a more complex head, but the change to soundscape-aligned validation and training. On the public leaderboard, the EfficientNet branch improved from an initial score of 0.632 to 0.711 after introducing soundscape validation, and then to 0.715 after adding augmentation and mixup. This showed that realistic validation design had a larger impact than architectural search alone.

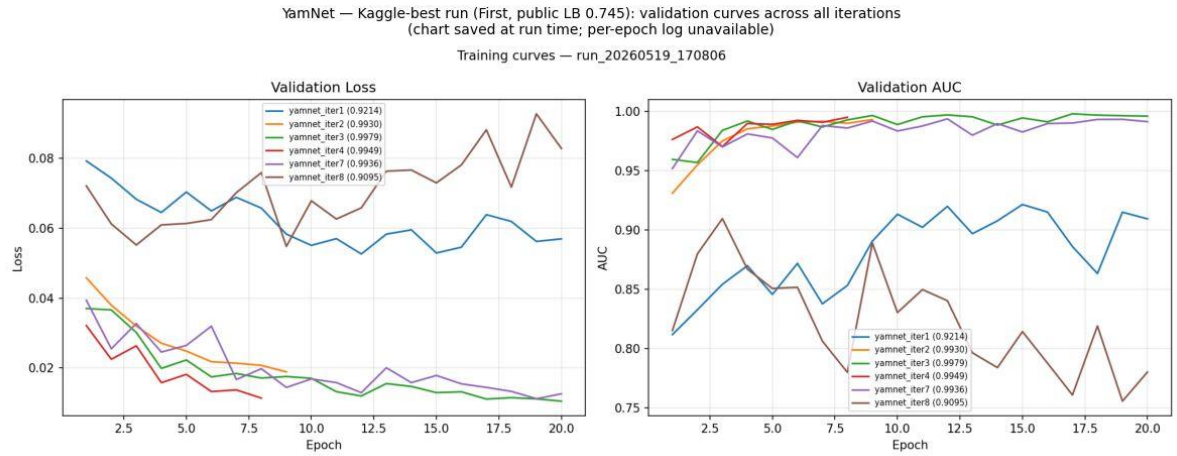


5.2 Backbone YAMNet

The second branch of the project used YAMNet as an audio-pretrained backbone. Unlike EfficientNet, which receives mel-spectrograms as image-like inputs, YAMNet operates directly on 16 kHz mono waveforms and produces 1024-dimensional audio embeddings. This made it attractive for the project because its pretraining on AudioSet is closer to the acoustic domain than ImageNet pretraining. The YAMNet SavedModel was also bundled locally, allowing the pipeline to run offline and remain compatible with the Kaggle submission environment.

The YAMNet agent precomputed embeddings for each clip, including several waveform-level augmentation variants, and then trained lightweight classification heads over the resulting sequence of embeddings. The LLM explored temporal Conv1D heads, depthwise convolutions, pooling strategies and attention-like structures. In practice, this branch was harder to stabilise than EfficientNet: many generated architectures failed due to shape mismatches, invalid Keras layer usage, or incorrect assumptions about the backbone output shape. These failures were still useful because they tested the agent's self-repair mechanism and exposed the importance of strict prompt constraints.

Once the pipeline was corrected, YAMNet produced competitive Kaggle results. Although some validation configurations were unstable, especially soundscape-based validation, the best YAMNet submission reached a public leaderboard score of 0.745, outperforming the EfficientNet submissions. This suggests that audio-domain pretraining was more valuable for real soundscape generalisation than higher offline validation AUC on focal clips. At the same time, the instability of the YAMNet branch shows that a better audio backbone alone is not enough: the agent also needs stronger architectural constraints and more reliable validation to guide its search.

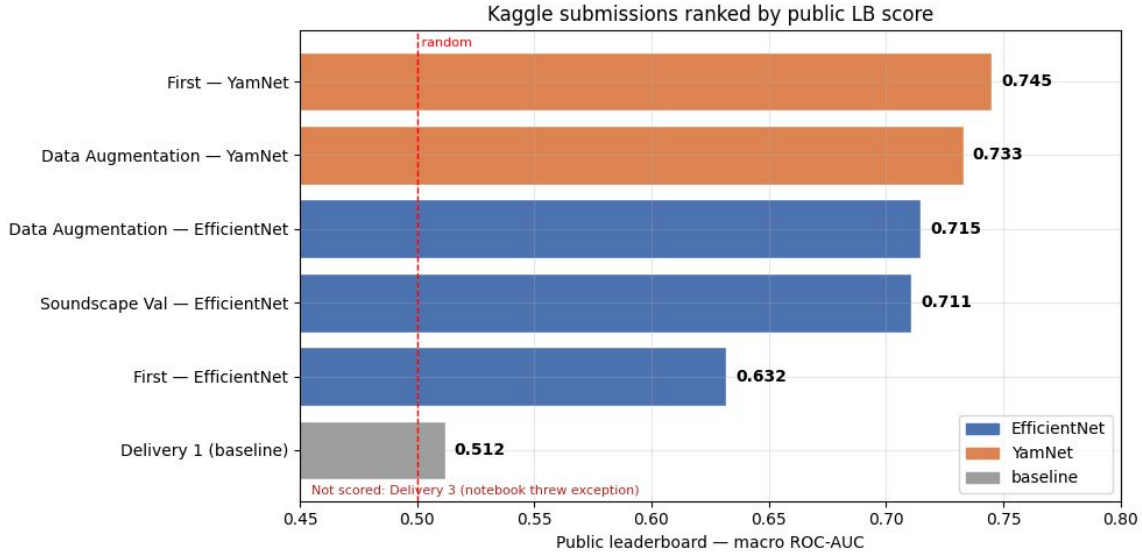


5.3 Comparison between models

Overall, the two backbones revealed different strengths. EfficientNet was the most stable branch during offline experimentation: it trained reliably, produced fewer invalid architectures, and reached the highest validation AUC, with some runs approaching 0.93. This made it useful for rapid architectural search and for analysing which classification heads were most effective. However, its validation scores were partly inflated by the mismatch between clean focal recordings and noisy soundscape test data.

YAMNet was less stable as an autonomous-agent target. The LLM frequently generated heads with invalid input shapes or unsuitable sequence operations, leading to several crash-and-retry cycles. Nevertheless, once the pipeline was corrected, YAMNet achieved the best public leaderboard score of the project, reaching 0.745 compared with the best EfficientNet score of 0.715. This suggests that domain-relevant audio pretraining was more important for real soundscape generalisation than achieving the highest offline validation AUC.

The main conclusion is that EfficientNet was better for controlled experimentation, while YAMNet was better aligned with the competition domain. EfficientNet demonstrated the value of the agent loop for architecture search, whereas YAMNet demonstrated the value of choosing a pretrained representation closer to the target data. In both cases, the largest gains came not only from the LLM-generated heads, but from improving the validation strategy, data augmentation, and robustness of the training pipeline.



6. Reflection on course content

6.1 Signal Preprocessing and Mel-Spectrograms

Raw audio is converted to 64-bin mel-spectrograms using librosa (32 kHz, 5 s, fmax = 16 kHz, hop length 512), yielding a (64, 313) image per clip. The mel scale emphasises lower frequencies, appropriate for bird vocalisations. Pre-computing all spectrograms into RAM eliminated disk I/O as the per-epoch bottleneck. The YAMNet variant bypassed this step entirely by accepting raw waveforms, illustrating that the spectrogram is an engineering choice rather than a universal requirement when an audio-pretrained backbone is available.

6.2 2D CNNs and Transfer Learning

EfficientNetB0 is the main backbone: a 2D CNN pretrained on ImageNet, used as a fine-tuned feature extractor for mel-spectrograms. The single-channel spectrogram is replicated to three channels so EfficientNet's RGB-trained weights can process it. Transfer learning from ImageNet to audio spectrograms is not a natural match — ImageNet images have spatial coherence and colour while spectrograms have frequency-time structure — but the approach works because early convolutional layers detect generic low-level features such as edges and textures that partially correspond to spectral patterns. The strong baseline AUC of 0.9047 on the first iteration, before any head search, confirms this.

6.3 Data Augmentation

Three audio-domain augmentations (Gaussian noise, random gain, time shift) and two spectrogram-domain augmentations (SpecAugment frequency and time masking) are applied during training only. The YAMNet agent extended this with pitch shift, time stretch, embedding-axis band masking, and mixup augmentation with Beta(0.4, 0.4) blending of sample pairs and soft labels to synthesise the multi-species overlap that characterises real soundscapes — directly targeting the domain shift between focal clips and field recordings.

6.4 Multi-Label Classification

The competition requires predicting the probability of presence for each of 234 species per 5-second window. Labels are encoded as multi-hot vectors (primary and secondary labels both set to 1). The output layer uses sigmoid activation and binary cross-entropy loss — the correct formulation for independent per-species predictions, not the softmax and categorical cross-entropy appropriate for mutually exclusive classes. This distinction proved practically important: the LLM incorrectly suggested switching to softmax after iteration 3 of Run_20260518, and the YAMNet branch initially compiled with the wrong AUC metric (micro-averaged instead of macro-averaged with num_labels=234), which produced trivially high and misleading validation scores.

6.5 Head Architectures Explored by the Agent

Across the nine experiments, the LLM proposed six structurally distinct head designs covering the course's main topics. Dense MLPs with dropout and batch normalisation appeared in most iterations and proved consistently competitive. MultiHeadAttention reshaped the flat 1280-d vector to a sequence of length 1 — architecturally valid but limited by the absence of positional structure, resulting in the lowest AUC of Run_20260518 at 0.9111. The Squeeze-and-Excitation block performed channel-wise recalibration (1280 → 80 → 1280 with ReLU/sigmoid) and trained the fastest. The Conv1D head treated the feature vector as a 1-step sequence and applied a 1280-filter convolution, achieving the second-best single-iteration result at 0.9178. Multi-pathway fusion concatenated three parallel processing paths. Grouped Feature Decomposition — splitting the 1280 dimensions into four independent groups — produced the best result at val_auc = 0.9237.

6.6 Effectiveness Summary

In decreasing order of observed impact: transfer learning from EfficientNetB0 was the single largest contributor, responsible for a baseline AUC of approximately 0.90 before any head search. Soundscape-based validation alignment contributed the largest leaderboard gain (+0.08). The architectural search by the agent contributed +0.019 val_auc within the clip-validation regime. SpecAugment and mixup augmentation were included in all runs and are standard practice for audio competitions. Among head designs, lightweight mechanisms (SE block, Conv1D, Grouped Decomposition) outperformed the full

MultiHeadAttention, consistent with the observation that the backbone's global average pooling discards temporal structure that attention would need.

7. Limitations and Future Work

7.1 What the Agent Could Not Do

The most fundamental limitation is the collapsed temporal axis. EfficientNetB0's global average pooling discards all temporal structure before the head sees any data — call timing, rhythm, and syllable sequence are irretrievably lost in a flat 1280-d vector. Conv1D and attention layers offered to the LLM operate on a pseudo-sequence of length 1 and cannot exploit this structure regardless of their complexity.

A related ceiling is the pretraining mismatch. EfficientNetB0 knows natural images, not bird vocalisations. YamNet partially addresses this, trained on AudioSet, but was not designed for fine-grained species discrimination and will conflate acoustically similar species that a bird-specific model would separate.

The in-domain validation set is too small to reliably guide the search. Only around 295 soundscape windows form the held-out split, so AUC differences below 0.01 are within the noise floor. The absence of recordist grouping means some apparent generalisation may reflect memorisation of background noise patterns rather than genuine species discrimination.

Finally, Gemma 4 (E4B) was also a bottleneck in the agent pipeline. Almost 20% of the iteration time was lost to crash-and-fix retries, and the model tended to propose shallow Dense + Dropout architectures unless explicitly prompted to explore more diverse designs. Additional limitations included the exclusion of rating = 0 recordings, the independent classification of each 5-second window, and the use of single-model inference without assembling or test-time augmentation.

7.2 Future Work

The highest-priority change is replacing EfficientNet with an audio-pretrained backbone. BirdNET is the strongest candidate given its taxonomy alignment with BirdCLEF; PANNs CNN14 and YAMNet are more readily deployable. Any of these would directly address the domain mismatch that currently limits both approaches.

Preserving the temporal axis is the next structural improvement. Setting pooling equal to None in the backbone would expose a time-resolved feature map, enabling Conv1D, GRU, or attention to model call timing and syllable structure, information that global average pooling currently discards entirely.

On the data side, background-noise mixing from real unlabelled soundscapes and pseudo-labelling the ~10,000 unlabelled soundscape files would expand in-domain training data substantially, without any additional annotation effort.

Finally, several low-cost gains require no retraining: run-level ensembling over the top-N stored models, test-time augmentation over overlapping 5-second windows, and replacing the single 80/20 split with recordist-grouped k-fold cross-validation for a more reliable validation signal.

8. Conclusion

This project delivers a working autonomous research agent that genuinely iterates and improves: it proposes deep learning architectures via a locally-hosted LLM, executes and trains them in a sandboxed environment, recovers from its own failures, logs every experiment in an append-only record, and produces Kaggle-ready inference artifacts. The central finding is that validation signal alignment, switching from focal-clip to soundscape-based validation, contributed more to leaderboard performance (+0.08) than the architectural search itself (+0.019 val_auc), confirming that distributional alignment between the training-time metric and the evaluation objective is the most important lever in transfer learning settings with significant domain shift. The autonomous-research paradigm proves most

valuable not as a replacement for engineering expertise but as a reproducible scaffold within which backbone, head search strategy, and validation protocol can each be upgraded independently.

9. Individual Contributions

All team members participated in all major stages of the project, including the discussion of ideas, definition of the agent architecture, interpretation of experimental results, debugging sessions, report planning, and final review of the repository. Although the work was collaborative throughout, each member had a primary area of responsibility.

Diogo Ribeiro focused mainly on the YAMNet branch of the project.

Pedro Fernandes focused mainly on the EfficientNet branch. This involved developing the mel-spectrogram pipeline.

Sofia Póvoa focused mainly on the exploratory data analysis, report development, and presentation materials.

Overall, the project was developed through continuous collaboration. Design choices, debugging decisions, interpretation of failures, and final conclusions were discussed collectively, ensuring that all team members understood the complete system rather than only their assigned component.

Video Presentation: https://youtu.be/O_03TRQfUfo